PROMPT ENGINEERING

Tecniche, Pattern e Strategie per comunicare con l'Intelligenza Artificiale

Michele Marchesi

marchesi@unica.it

Corso di Ingegneria del Software
A.A. 2025-26

Il Nuovo "Coding"

- L'Intelligenza Artificiale Generativa (GenAI) sta rivoluzionando il nostro modo di lavorare.
- Modelli come GPT-5, Gemini, Claude e DeepSeek sono strumenti potentissimi, ma la loro qualità dipende da un fattore cruciale: **il prompt**.
- Il prompt engineering è spesso definito come il "***nuovo coding***": la competenza fondamentale per interagire e dirigere l'IA.

Large Language Models (LLM)

- Un LLM è un tipo di intelligenza artificiale progettato per comprendere, generare e manipolare il linguaggio umano
- Sono addestrati su enormi volumi di dati testuali (libri, articoli, pagine web, codice) per apprendere pattern, regole grammaticali e relazioni semantiche
- Non "pensano" come gli umani, ma sono motori predittivi statistici che analizzano il contesto per prevedere la parola (o token) più probabile successiva

Il Concetto Fondamentale: Il Token

- La prima operazione di un LLM è la tokenizzazione
- I token sono le unità atomiche in cui il testo viene suddiviso per essere elaborato dalla macchina
- Un token può essere:
  - Una parola intera (es. *gatto*)
  - Una sottoparola (es. *anti*, *zione*, *ente*)
  - Un singolo carattere (es. *a, ;, ?, !*)
  - Un byte
- A ogni token del dizionario (vocabolario) del modello viene associato un numero ID univoco. Il modello lavora con questi numeri, non con le parole .

Come Avviene la Tokenizzazione

- Non si usano solo parole intere perché parole nuove o inventate (es. "instagrammabile") sarebbero sconosciute (problema OOV - Out of Vocabulary), e inoltre ciò richiederebbe un vocabolario immenso.
- La soluzione: Tokenizzazione per sottoparole (Subword Tokenization):
  - È il metodo più usato oggi (es. BPE, WordPiece)
  - Bilancia dimensione del vocabolario e capacità di gestire parole rare.
  - Le parole comuni restano intere (the, cat). Le parole rare vengono scomposte in pezzi più piccoli e frequenti: (un + watch + ed)
- Ad esempio: la parola "antidisestablishment" potrebbe diventare: anti + dis + establish + ment

Il Cuore dell'Architettura: Il Transformer

- La stragrande maggioranza degli LLM moderni si basa sull'architettura Transformer .
- Innovazione chiave: Il meccanismo di Self-Attention
- Permette al modello di valutare l'importanza di ogni parola rispetto a tutte le altre nella frase, indipendentemente dalla distanza .
- Esempio: Nella frase "*I went to the bank to deposit my paycheck, but it was closed because the river bank had flooded the road*", il modello capisce che il primo "bank" (banca) è correlato a "deposit" e "paycheck", mentre il secondo "bank" (riva) è correlato a "river" e "flooded", grazie al contesto.
- A differenza dei vecchi modelli (RNN), i Transformer elaborano i dati in parallelo, rendendo l'addestramento su enormi quantità di dati molto più efficiente

La Fase di Addestramento (Training)

1) **Pre-addestramento (Pre-training)**: Il modello "legge" immense quantità di testo non etichettato (petabyte). Impara prevedendo la parola successiva in una frase (apprendimento auto-supervisionato). È la fase più costosa (milioni di dollari e migliaia di GPU).
2) **Ottimizzazione (Fine-tuning)**: Il modello base viene specializzato. Lo si addestra su un dataset più piccolo e specifico per seguire istruzioni o svolgere compiti precisi (es. fare il chatbot).
3) **Apprendimento con Feedback Umano (RLHF)**: Si affina ulteriormente il modello utilizzando valutatori umani che premiano le risposte migliori, rendendo l'output più allineato alle preferenze umane.

Pre-elaborazione e Contesto

- La pre-elaborazione consiste principalmente nella tokenizzazione e nella conversione dei token in embedding (vettori numerici) che il modello può elaborare
- Il concetto di "contesto" è vitale. I modelli hanno una "finestra di contesto" (context window), ovvero il numero massimo di token che possono considerare in una volta
- I modelli moderni arrivano a milioni di token di contesto.
- Problema: tokenizzazioni diverse possono dare risultati diversi. Ad esempio, uno spazio in più o una lettera maiuscola possono cambiare la tokenizzazione e quindi la risposta del modello .

Post-elaborazione

- Dopo che il modello ha generato la sequenza di token, questi vengono riconvertiti in testo leggibile (detokenizzazione) .
- LLM Specializzati per il Coding:
  - Sono addestrati su vaste basi di codice sorgente e documentazione tecnica
  - Sono utilizzati per: completamento automatico del codice, refactoring, debugging, spiegazione di funzioni, generazione di test, scrittura di documentazione
  - Esempi: Codex (base di GitHub Copilot), Codestral (Mistral), Claude Coder, StarCoder (open-source)
  - Criteri di scelta: precisione del codice, integrazione con l'IDE, sicurezza dei dati (possibilità di esecuzione locale)

Limitazioni degli LLM

- Gli LLM sono macchine statistiche: non capiscono il significato o la logica come un essere umano
- Allucinazioni: gli LLM endono a inventare fatti **con sicurezza** quando non conoscono la risposta .
- Bias (Pregiudizi): gli LLMossono riprodurre e amplificare pregiudizi e viste di parte presenti nei dati di addestramento
- Costo Energetico: l'addestramento e l'uso di questi modelli richiedono un'enorme potenza di calcolo, con impatti ambientali non trascurabili
- Mancanza di aggiornamenti: la conoscenza è "congelata" alla data dell'addestramento, a meno che non vengano integrati con strumenti di retrieval (Retrieval-Augmented Generation - RAG)

I principali LLM per sviluppo software

Società	Nazione	Introduzione	Ultima versione	Versione gratis	Vers. a pagamento	Popolarità	Qualità
GitHub (Microsoft)	USA	2021	v1.109 (2026)	Sì (2000 completamenti/ mese per utenti verificati)	Pro ($10/mese), Business ($19/ utente), Enterprise ($39)	Altissima - il più adottato globalmente	Eccellente per workflow GitHub
OpenAI	USA	2020 (GPT-3)	GPT-5.3-Codex (2026)	Sì (gratuito con limiti)	ChatGPT Plus, Team, Enterprise	Altissima - 100M+ utenti attivi/ settimana	Eccellente - leader in coding specializzato
Google	USA	2023 (Bard)	3.1 Pro (2026)	Sì (tramite AI Studio)	API: $2/milione token	Altissima	Eccellente - 1° in 13/16 benchmark
Anthropic	USA	2023 (Claude)	Opus 4.6 (2026)	No (solo a pagamento)	Pro ($20/mese), API, Enterprise	Molto alta	Eccellente
Amazon AWS	USA	2022	2026	Sì (Individuale gratis)	Professionale ($19/mese)	Alta - tra i più usati per sviluppo AWS	Buono - ottimo per integrazione AWS
Anysphere	USA	2023	Long-running Agents (feb 2026)	Sì (Hobby tier con limiti)	Pro ($20), Pro+ ($60), Teams ($40/utente)	Molto alta - apprezzato per repository complessi	Eccellente - agenti autonomi per task complessi
Tabnine	Israele	2017	CLI Agent (2026)	Sì (base)	Enterprise ($39+/mese)	Alta - leader per enterprise privacy	Eccellente per ambienti air-gapped

I principali LLM per sviluppo software

Società	Nazione	Introduzione	Ultima versione	Versione gratis	Vers. a pagamento	Popolarità	Qualità
Codeium (acquisita da Google)	USA	2022	2026	Sì (illimitato)	Teams (collaborative features)	Alta - tra sviluppatori individuali	Buono - solido per autocomplete
Hugging Face	USA	2016	2026	Sì (modelli open)	Enterprise (personalizzata)	Altissima - hub di riferimento modelli	Eccellente - standard industria NLP
DeepSeek (深度求索)	Cina	2024	V3.2 (2025)	Sì (modelli open source)	API a consumo (via piattaforme cloud)	Emergente - crescente adozione globale	Buono - competitivo con modelli USA
Alibaba (阿里巴巴)	Cina	2023	Qwen 2.5 2026	Sì (personale gratuito)	Enterprise (da 79/utente/anno)	Altissima - download >10M, #1 in Cina	Eccellente - leader in Java/Go e integrazione Alibaba Cloud
Baidu (百度)	Cina	2024	3.5S (2026)	Sì (personale gratuito)	Enterprise (personalizzata)	Altissima - IDC 8/9 dimensioni full marks	Eccellente - C++ #1, engineering excellence
Tencent (腾讯)	Cina	2025 (Craft)	Craft v2 (2026)	Sì (personale gratuito)	Enterprise (78/utente/anno)	Alta - integrazione ecosistema Tencent	Ottimo

Che cos'è il Prompt Engineering?

- Prompt Engineering:
  - **Definizione**: L'arte e la scienza di progettare input (***prompt***) efficaci per i modelli linguistici (LLM).
  - **Non è programmazione**: Non si segue una sintassi rigida, ma si usano istruzioni in linguaggio naturale.
  - **Obiettivo**: Guidare l'AI a produrre output specifici, accurati e utili.
  - **Contesto**: Strumenti come ChatGPT, Claude, Gemini, Co-pilot sono accessibili a tutti, ma i risultati dipendono dalla qualità del prompt.
- Il prompt engineering non richiede conoscenze tecniche di coding, ma una buona comprensione della logica e del lin-guaggio. È il "ponte" tra l'intento umano e la risposta della macchina

Definizione accurata

- Il Prompt Engineering è il processo strategico di progettazione, perfezionamento e ottimizzazione del testo di input (il prompt) per guidare i modelli di IA generativa verso la produzione di output accurati, pertinenti e di alta qualità.
- Non si tratta di modificare il modello o i suoi dati di addestramento, ma di "parlare la sua lingua" per ottenere il massimo delle sue potenzialità.
- In sintesi: è l'arte di formulare la domanda giusta per ottenere la risposta desiderata.

Importanza del Prompt Engineering

- Un buon prompt trasforma una query vaga in un'istruzione precisa.
- **Qualità dell'output**: Influenza direttamente l'accuratezza, la pertinenza e la coerenza delle risposte. Istruzioni di qualità producono risultati di qualità.
- **Efficienza**: Riduce la necessità di correzioni manuali e post-elaborazione, risparmiando tempo e risorse.
- **Controllo**: Permette di definire tono, stile e formato dell'output, allineando l'IA alle nostre esigenze specifiche.

Come funziona il Prompt Engineering

- I modelli di IA sono basati su architetture complesse (transformer) che elaborano il linguaggio naturale.
- Il prompt engineering colma il divario tra l'intenzione umana (spesso ambigua) e l'esecuzione della macchina (che necessita di istruzioni chiare).
- Un prompt ben congegnato fornisce il contesto necessario e riduce l'ambiguità, guidando il modello verso l'output desiderato.
- Il ponte tra "Intenzione Umana" e "Output della Macchina" è costruito dal "Prompt Engineering"

Lo scopo del Prompt Engineering

- L'obiettivo fondamentale è **massimizzare l'utilità e l'affidabilità dei modelli di IA**
- **Obiettivi primari:**
  - **Migliorare la qualità**: Output più accurati, pertinenti e utili.
  - **Ridurre l'ambiguità**: Minimizzare interpretazioni errate da parte del modello.
  - **Aumentare l'efficienza**: Ottenere il risultato desiderato in meno iterazioni.
  - **Garantire consistenza**: Mantenere stile, tono e formato uniformi in output multipli.
  - **Prevenire allucinazioni**: (informazioni false) e pregiudizi indesiderati.
  - **Facilitare l'integrazione**: Produrre output strutturati (JSON, XML) per uso in applicazioni software.

Esempio 1 - Dalla Vaghezza alla Precisione

- Prompt Vago (Cosa NON fare): > *"Scrivimi qualcosa sull'intelligenza artificiale."*
  - Risultato probabile: Un testo generico, prolisso e poco utile.
- Prompt Ingegnerizzato (Cosa fare): > *"Sei un divulgatore scientifico. Devi scrivere un post per LinkedIn di massimo 150 parole che spieghi i benefici del prompt engineering per i project manager. Il tono deve essere professionale ma accessibile. Includi un esempio pratico e termina con una domanda per coinvolgere i lettori."*
  - Risultato: Un contenuto mirato, con un tono definito, una struttura chiara e un obiettivo preciso (engagement).

Esempio 2 - Controllo del Formato (Dati)

- Prompt Senza Indicazioni: > *"Quali sono i tre libri più venduti di sempre e chi li ha scritti?"*
  - Risultato: Una risposta in un paragrafo discorsivo, difficile da estrapolare velocemente.
- Prompt Ingegnerizzato (con focus sul formato): > *"Fornisci una lista dei tre libri più venduti di sempre. Per ogni libro, crea una tabella con le seguenti colonne: Titolo, Autore, Anno di Pubblicazione, Genere. Ordina la tabella per copie vendute in ordine decrescente."*
  - Risultato: Una tabella pronta per essere copiata in un foglio di calcolo o in un report, estremamente fruibile.

Esempio 3 - Controllo del Formato (Dati)

- Prompt Senza Indicazioni: > *"Dammi informazioni su questi prodotti: laptop, mouse, tastiera."*
  - Risultato: Una risposta in un paragrafo discorsivo, difficile da estrapolare velocemente.
- Prompt Ingegnerizzato (con focus sul formato): > *"Sei un assistente e-commerce. Per ciascuno dei seguenti prodotti (laptop, mouse, tastiera), genera una scheda con: nome, categoria, prezzo medio (EUR), 3 pro, 3 contro, link a recensione affidabile. Output in formato JSON valido con chiave 'prodotti'."*
  - Risultato (vedi slide seguente): un file JSON pronto per essere elaborato da un programma, estremamente fruibile.

Esempio 3 - Controllo del Formato (Dati)

```
{
  "prodotti": [
    {
      "nome": "Laptop XYZ",
      "categoria": "Informatica",
      "prezzo_medio_eur": 899,
      "pro": ["Leggero", "Batteria 12h", "Garanzia 3 anni"],
      "contro": ["Schermo non touch", "RAM non espandibile",
"Ventola rumorosa"],
      "recensione_link": "https://esempio.it/recensione-xyz"
    },
   . . .
  ]
}
```

Tecniche Fondamentali

- Esistono diverse strategie per interagire con i LLM, ognuna adatta a scopi diversi.
- Le tecniche vanno dalla più semplice (zero-shot) a quelle più complesse (chain-of-thought) per gestire compiti di ragionamento articolati.
- Nel seguito, illustreremo le tecniche più importanti, dandone caratteristiche, campi applicativi ed esempi

Zero-Shot Prompting

- Richiesta eseguita senza fornire esempi preliminari.
- Ideale per: richieste semplici e dirette che il modello dovrebbe già conoscere (es. traduzione, classificazione semplice, generazione di idee) e per esplorazione iniziale.
- Vantaggi: rapido, minimale, buono per brainstorming.
- Limiti: poco controllo sul formato o stile dell'output.
- Esempio:
    - "Traduci in francese: 'Buongiorno, come stai?'

Few-Shot Prompting

- Si forniscono al modello alcuni esempi (shot) di input e output desiderato all'interno del prompt. Questo aiuta il modello a “capire” il pattern e il contesto del compito.
- Ideale per: richieste con formato specifico, classificazione, trasformazioni strutturate.
- Vantaggi: Maggiore controllo, riduzione delle ambiguità, apprendimento per imitazione.
- Limiti: Consuma più token, richiede preparazione degli esempi.

Few-Shot Prompting: struttura

- Shot 1: Input -> Output
- Shot 2: Input -> Output
- Richiesta: Nuovo Input -> ?

- Es:
  > Trasforma le frasi dal registro informale a quello formale in italiano.
  > Input: 'Ciao, come stai?' -> Output: 'Salve, come sta?'
  > Input: 'Arrivederci' -> Output: 'Arrivederla.'
  > Ora trasforma: Input: 'Grazie mille.'
  RISULTATO: >*La ringrazio molto*

Chain-of-Thought (CoT)

- Si incoraggia il modello a scomporre un problema complesso in una serie di passaggi intermedi (una catena di ragionamento) prima di fornire la risposta finale.
- Ideale per: Problemi logici, matematica, decisioni complesse, debugging.
- Vantaggi: Maggiore accuratezza, trasparenza nel processo, identificazione errori.
- Limiti: Output più lunghi, possibile sovra-ingegnerizzazione per task semplici.
- Prompt tipico: "*Prima di rispondere, spiega il tuo ragionamento passo dopo passo. Alla fine, fornisci la risposta finale*."

Chain-of-Thought (CoT): esempio

- Risoluzione di un problema di logica.
- Prompt diretto:
- *"Se 5 macchine producono 5 gadget in 5 minuti, quanto tempo impiegano 100 macchine per produrre 100 gadget?"*
- Prompt con CoT:
- *"Risolvi questo problema passo dopo passo. Spiega ogni passaggio logico. Alla fine, dai la risposta numerica. Problema: Se 5 macchine producono 5 gadget in 5 minuti, quanto tempo impiegano 100 macchine per produrre 100 gadget?"*
- Risposta attesa con CoT:
  1. Ogni macchina produce 1 gadget ogni 5 minuti.
  2. Quindi 100 macchine producono 100 gadget nello stesso tempo: 5 minuti.
- Senza CoT, molti modelli rispondono "*100 minuti*".

Role Prompting (o Persona Prompting)

- Si assegna al modello un ruolo o una "persona" specifica da interpretare durante la generazione della risposta.
- Vantaggi: di determina il tono, lo stile, il linguaggio e la prospettiva dell'output, rendendolo più coinvolgente e adatto al contesto.
- Limiti: Output più lunghi.
- Esempio: "*Agisci come un avvocato specializzato in privacy. Spiega a un cliente, usando un linguaggio semplice, cosa implica il nuovo regolamento sulla protezione dei dati per la sua piccola impresa.*"

Altre tecniche

- **Generated Knowledge**: chiedere al modello di generare fatti/conoscenze sul tema prima di rispondere alla domanda principale.
- **Self-Consistency**: generare multiple risposte allo stesso prompt e selezionare la più frequente/coerente. È utile per ridurre allucinazioni, aumentare affidabilità in task critici.
- **Prompting Negativo**: Specificare cosa **non** si vuole vedere nell'output.
- **RAG (Retrieval-Augmented Generation)**: Fornire al modello contesto esterno (es. documenti aziendali) da cui attingere per rispondere, riducendo le allucinazioni.
- **Generazione Strutturata**: Chiedere esplicitamente output in formati come JSON o XML per una facile integrazione con altri sistemi.

Esempio di Generated Knowledge

- Oggetto: Preparare una breve presentazione sulle energie rinnovabili.
- Prompt a due fasi:
  FASE 1 - Genera conoscenza:
  "Elenca 5 fatti verificati e recenti (2023-2024) sull'energia solare fotovoltaica, con fonte attendibile per ciascuno."

  [Il modello genera i fatti]

  FASE 2 - Usa la conoscenza:
  "Usando i fatti sopra, crea una slide di presentazione con: titolo accattivante, 3 punti chiave, 1 dato sorprendente, 1 call-to-action. Formato: markdown per slide."
- Vantaggio: L'output è basato su informazioni strutturate, non su conoscenze generiche del modello.

Pattern di Prompting Comuni

- Se le tecniche sono le "strategie generali", i pattern sono modelli di prompt riutilizzabili e specifici per risolvere categorie di problemi ricorrenti.
- Sono come dei "template intelligenti" che possiamo adattare alle nostre esigenze.
- Categore di pattern:
  - **Approcci *Learning-Driven***: Zero-shot, few-shot.
  - **Metodi focalizzati sull'interazione:** Prompt interattivi e raffinamento iterativo.
  - **Pattern per compiti specifici**: Ottimizzati per coding, esecuzione di riassunti, ecc.
  - **Ottimizzazione di modelli**: Regolazione di parametri come la temperatura.

Pattern Template (Output Customization)

- Obiettivo: Garantire che l'output dell'IA segua sempre una struttura predefinita e specifica.
- Struttura del pattern:
  1. Si fornisce un template con dei “segnaposto” (placeholders).
  2. Si chiede al modello di riempire i segnaposto con i contenuti, preservando la formattazione.
- Utilità: Perfetto per generare report, schede prodotto, riassunti strutturati in modo uniforme.

Pattern Template - Esempio

- Prompt: "*Ti fornirò un template per l'output. Le parole in MAIUSCOLO sono i miei segnaposto. Mantieni la formattazione. Il template è:*

  Nome Prodotto: [NOME] Categoria: [CATEGORIA]
  Prezzo: €[PREZZO] Caratteristiche principali:
  - [CARATTERISTICA_1]
  - [CARATTERISTICA_2]
  - [CARATTERISTICA_3]
  - Descrizione breve: [UNA FRASE]

  Ora, compila il template per il prodotto 'MacBook Pro M3'."

Pattern Persona-Ruolo

- Obiettivo: struire il modello a "interpretare" un esperto, un personaggio o un ruolo specifico..i
- Struttura del pattern:
    1. "Agisci come [ruolo specifico]."
    2. "Il tuo obiettivo è: [obiettivo].
    3. "Il tuo tone è: [*professionale, introduttivo*,...]
- Utilità: Attiva conoscenze e stili linguistici associati a quel ruolo nel training del modello.
- Ambiti applicativi: comunicazione efficace in team multidisciplinari, divulgazione, supporto clienti.

Pattern Persona-Ruolo - Esempio

- *Comportati come un computer vittima di un attacco informatico.*
  Rispondi a qualsiasi cosa digiti con l'output che produrrebbe il terminale Linux. Chiedimi il primo comando.

- *Comportati come l'agnello della filastrocca "Mary had a little lamb". Io ti dirò cosa sta facendo Mary e tu mi dirai cosa sta facendo l'agnello.*

- *Comportati come uno chef gourmet. Io ti dirò che cosa mi piace e tu mi suggerirai dei piatti adeguati.*

Pattern Persona-Audience

- Obiettivo: adattare la spiegazione di un concetto complesso al livello di conoscenza di uno specifico interlocutore che ascolta (Audience).
- Struttura del pattern:
  1. "Spiegami [concetto X]."
  2. "Supponi che io sia [ruolo o livello di conoscenza Y]."
- Utilità: comunicazione efficace in team multidisciplinari, divulgazione, supporto clienti.

Pattern Persona-Audience - Esempio

- Scopo: spiegare i buchi neri
- Prompt per l'Astrofisico: > "*Spiegami il concetto di 'orizzonte degli eventi di un buco nero'. Supponi che io sia un ricercatore astrofisico.*" > Output: Ricco di dettagli tecnici, gergo specialistico, equazioni.
- Prompt per il Bambino: > "*Spiegami il concetto di 'orizzonte degli eventi di un buco nero'. Supponi che io sia un bambino di 8 anni.*" > Output: Usa metafore semplici (es. "un punto di non ritorno, come una cascata da cui non si può risalire"), linguaggio semplice, evita tecnicismi.

Pattern Cognitive Verifier

- Obiettivo: migliorare l'accuratezza di una risposta a una domanda complessa, facendo sì che l'IA generi prima una serie di domande di approfondimento.
- Struttura del pattern:
  1. Quando ti viene posta una domanda, genera domande aggiuntive che aiuterebbero a rispondere in modo più accurato.
  2. Combina le risposte a queste domande per formulare la risposta finale.
- Utilità: diagnosi mediche, analisi di problemi tecnici, consulenza strategica.

Pattern Cognitive Verifier - Esempio

- Prompt (utente): "*Qual è il periodo migliore per visitare il Giappone? Prima di rispondere fammi domande aggiuntive per aiutarti a rispondere meglio.*"
- IA (applicando il pattern): "*Per darti una risposta più accurata, ho bisogno di alcune informazioni*:
    1. *Quali sono le tue principali attività di interesse (es. sciare, ammirare i ciliegi in fiore, escursionismo, visitare città)?*
    2. *Preferisci evitare la folla o non è un problema?*
    3. *Hai un budget più alto per l'alta stagione o cerchi un'esperienza più economica? In base alle tue risposte, potrò consigliarti il periodo migliore per te."*

Pattern Reflection & Iterative Refinement

- Obiettivo: far Criticare e Migliorare l'AI
- Reflection Pattern: chiedere al modello di valutare e migliorare la propria risposta.
- Iterative Refinement: eseguire più cicli di feedback per affinare l'output.
- Struttura del pattern:
  1. Genera una prima bozza.
  2. Chiedi: "Quali sono i 3 punti deboli di questa risposta? Come li miglioreresti?"
  3. Rigenera la versione migliorata.
- Utilità: output più accurati e completi, riduzione di errori e omissioni, maggiore allineamento con le aspettative.

Pattern Menu Actions

- Obiettivo: trasformare l'IA in uno strumento interattivo con comandi predefiniti, simile a un menu di applica-zioni.
- Struttura del pattern:
  1. "Ogni volta che scrivo [comando X], tu esegui [azione Y]."
  2. Si possono definire più comandi.
  3. Alla fine di ogni azione, chiedi all'utente la prossima azione.
- Utilità: assistenti per lo sviluppo software, creazione di contenuti in serie, automazione di flussi di lavoro.

Pattern Menu Actions - esempio

- *> Ogni volta che digito "aggiungi CIBO", aggiungerai CIBO alla mia lista della spesa e aggiornerai la mia stima della spesa.*
  Ogni volta che digito "rimuovi CIBO", rimuoverai CIBO dalla mia lista della spesa e aggiornerai la mia stima della spesa.
  Ogni volta che digito "risparmia", elencherai alternative al CIBO che ho aggiunto per risparmiare denaro.
  Alla fine, mi chiederai l'azione successiva. Chiedimi la prima azione.

Pattern Flipped Interaction & Calendar

- **Flipped Interaction:**
- Invece di dare tutte le istruzioni, chiedi all'AI di farti domande per chiarire il task.
- Esempio: "*Prima di scrivere il report, fammi 3 domande per capire meglio le mie esigenze.*"

- **Calendar/Step-by-Step**:
- Scomporre task complessi in una sequenza temporale o logica di sotto-task.
- Esempio: "*Pianifica un progetto in 4 settimane: settimana 1: ricerca, settimana 2: bozza, ecc.*"

Altri Pattern

- The Question Refinement Pattern
- The Game Play Pattern
- The Meta Language Creation Pattern
- The Recipe Pattern
- The Alternative Approaches Pattern
- The Ask for Input Pattern
- The Fact Check List Pattern
- The Tail Generation Pattern
- The Format of the Semantic Filter Pattern

Esempi pratici: Pattern Template

- Template per Recensioni Prodotti:
  Scrivi una recensione per [PRODOTTO] destinata a [TARGET].
  Struttura:
  1. Titolo accattivante (max 10 parole)
  2. Pro (3 punti, max 15 parole ciascuno)
  3. Contro (2 punti, max 15 parole ciascuno)
  4. Voto finale (1-10) con breve motivazione
  Tono: [TONO]. Evita: [ELEMENTI_DA_EVITARE].
- Pattern **Template**:
  - Usare una struttura fissa con spazi vuoti da riempire.
  - Esempio: "*Scrivi una email per [Destinatario] riguardo [Argomento] con tono [Tono].*"

Esempi pratici: Pattern Reflection

- >*"Genera un report di 300 parole sull'andamento Q1 del team vendite".*
- L'A.I risponde con: [Output iniziale].
- >*"Ora, agisci come revisore senior: identifica 2 punti da approfondire e 1 affermazione da verificare con dati. Riscrivi il report integrando questi miglioramenti."*
- L'A.I risponde con: [Output rivisto].
- . . . si può continuare

Altri esempi pratici

Prompt Debole	**Prompt Ingegnerizzato**
"Scrivi una email di vendita."	"Agisci come venditore senior. Scrivi una email fredda per un software HR. Target: CEO di PMI. Tono: professionale ma empatico. Includi una proposta di call per una demo."
"Cos'è la fotosintesi?"	"Spiega la fotosintesi come se parlassi a un bambino di 10 anni. Usa un'analogia con una fabbrica."
"Riassumi questo articolo." <Articolo allegato>	"Estrai i 5 punti chiave di questo articolo in un elenco puntato. Evidenzia eventuali dati statistici citati." <Articolo allegato>

Best Practices e Consigli

- Siate Chiari e Specifici: Evitate ambiguità.
- Fornite Contesto: Più informazioni date, migliore è il risultato.
- Specificate il Formato atteso: (JSON, Tabella, Elenco, Codice).
- Iterate: Il primo prompt raramente è perfetto.
- Delimitatori: Usate virgolette o triple virgolette """ per separare le istruzioni dai dati.
- Trattate l'AI come un collaboratore intelligente ma che non legge nel pensiero. Siate espliciti.

Checklist Operativa

- Prima di inviare il prompt:
  - [ ] Ho definito chiaramente l'obiettivo?
  - [ ] Ho specificato formato, tono e lunghezza?
  - [ ] Ho fornito contesto sufficiente?
  - [ ] Ho usato delimitatori per separare istruzioni e dati?
  - [ ] Ho considerato se servono esempi (few-shot)?

- Dopo aver ricevuto la risposta:
  - [ ] Ho verificato accuratezza e fonti?
  - [ ] L'output è utilizzabile così com'è o serve raffinamento?
  - [ ] Posso salvare questo prompt come template per il futuro?

Errori da Evitare

- **Prompt troppo vaghi:** "*Parlami di X...*"
- **Prompt troppo lunghi e confusi**: L'AI può perdere il filo (perdita di attenzione).
- **Chiedere troppe cose insieme**: Meglio suddividere task complessi in più prompt.
- **Fidarsi ciecamente**: Verificare sempre i fatti (l'AI può allucinare).
- **Ignorare i limiti del modello**: Non chiedere accesso a dati in tempo reale se il modello non ha strumenti web.
- **Non iterare**: Il primo prompt raramente è perfetto; affinarlo è parte del processo.

Conclusioni

- Il prompt engineering è una competenza apprendibile: pratica e metodo battono il talento innato.
- Strutturare il prompt (ruolo, contesto, vincoli, formato) è più importante della lunghezza.
- Tecniche e pattern sono "ricette" collaudate: usarle, adattarle, combinarle.
- L'iterazione e la verifica umana restano essenziali: l'AI è un collaboratore, non un sostituto.

Queste slide sono state create correggendo e fondendo insieme due presentazioni su “Prompt Engineering: tecniche e pattern” e una su “Introduzione agli LLM”, generate dagli LLM DeepSeek e Qwen.